



GROUP 28 · CSIT321

System Architecture

Certacito.ai · A4 Prototype Presentation appendix

Project Certacito.ai, AI agent governance platform

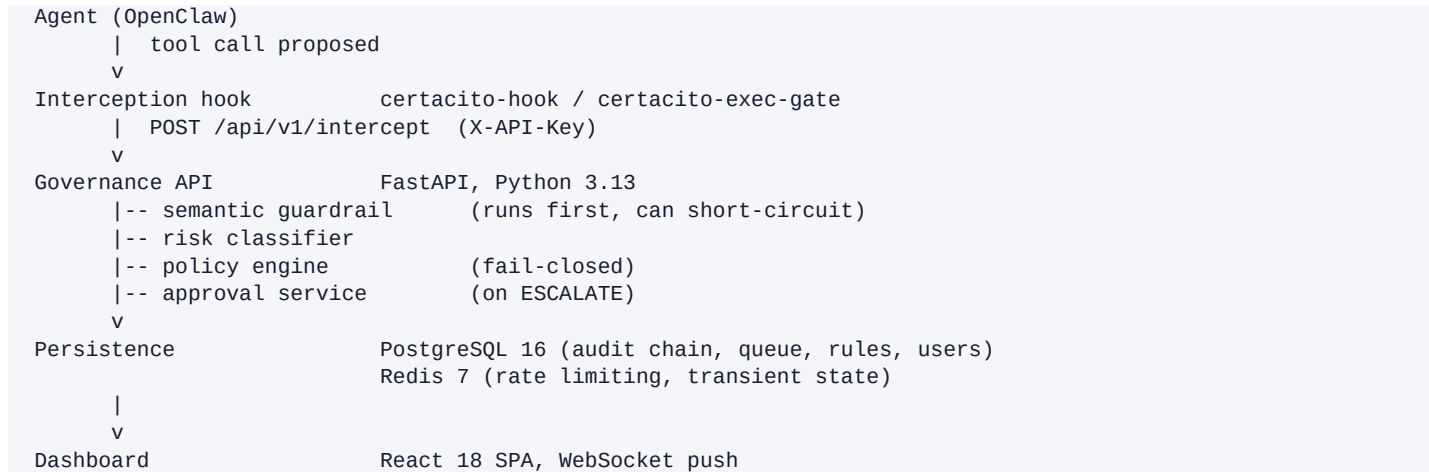
Document architecture.md

Date 07 August 2026

Live system <http://20.92.93.30>

Certacito.ai is a governance layer that sits between an AI agent and the actions it wants to take. Every tool call is intercepted before execution, evaluated against policy that lives outside the model, and recorded in a tamper-evident log. This describes what is actually built and deployed, not a target state.

The layers



Front end

React 18 with TypeScript, built by Vite 6.3. Tailwind CSS 4 for styling, Recharts for the dashboard visualisations, Radix primitives underneath the component set inherited from the A3 Figma work.

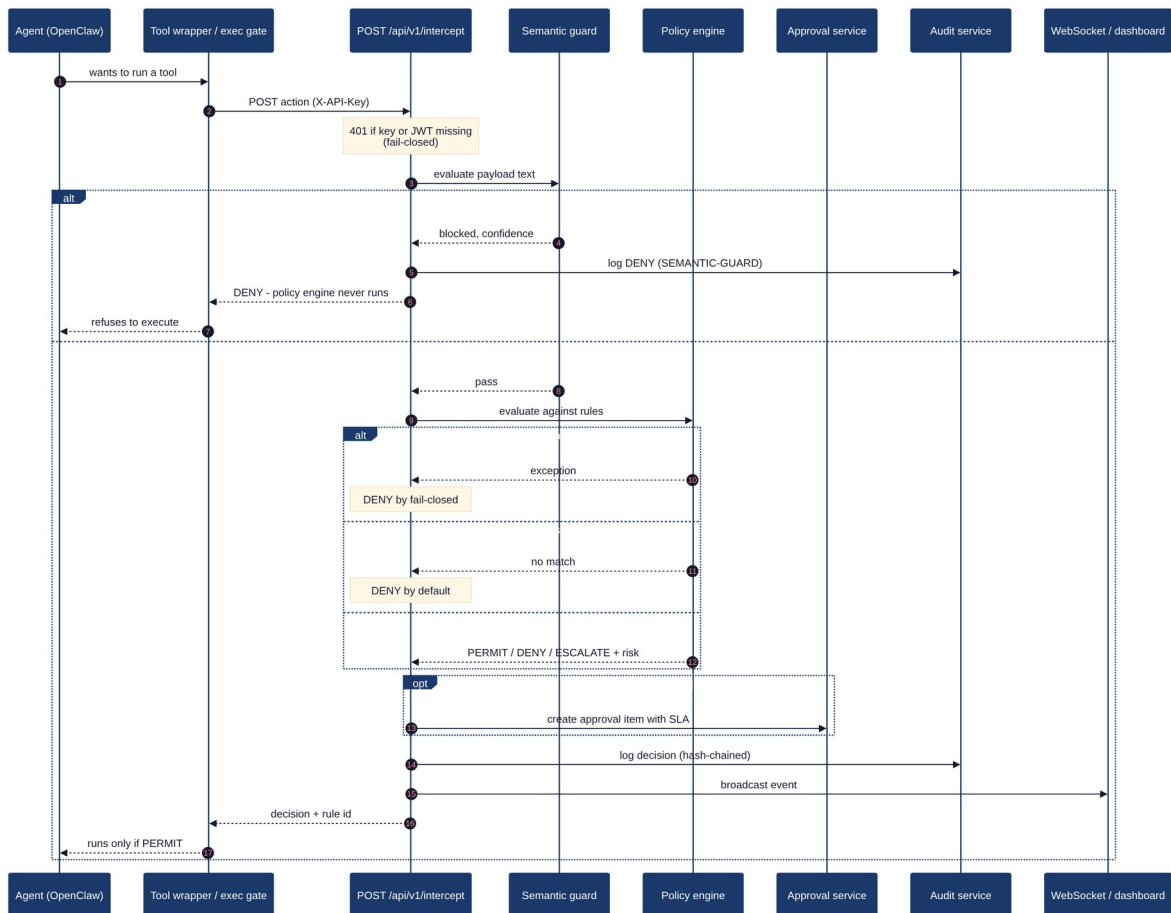
The SPA is a single bundle. `API_BASE` resolves to `import.meta.env.VITE_API_URL || ""`, so in the deployed configuration the frontend calls the same origin it was served from and needs no build-time API URL.

Live updates come over a WebSocket at `/api/v1/ws/live` rather than polling. The socket authenticates with `?token=<jwt>`; an unauthenticated socket is rejected.

Polling exists only as a fallback if the socket cannot connect.

The interception path, step by step

The core governance decision, traced from `backend/api/interception.py`. Note the three separate DENY branches: the semantic guard short-circuits before the policy engine runs, an engine exception denies, and no matching rule denies.



Back end

FastAPI on Python 3.13, served by uvicorn. Fourteen routers mounted under `/api/v1`: interception, audit, approval, policy, auth, stats, guardrails, websocket, agents, demo, reports, dryrun, trends and API keys.

Two pieces of middleware sit in front of every request: CORS, and a rate limiter allowing 60 requests/minute generally and 10/minute on auth endpoints. Behind a reverse proxy the limiter keys off `X-Real-IP`, because without it every client shares the proxy's bucket and one active dashboard locks everyone out.

Static files are mounted last, at `/`, so `/api/*` and `/health` keep winning against the SPA catch-all.

The agent side

Certacito governs an agent that lives outside this repository. That separation is the design, not an omission: the governance layer holds the policy, the audit chain and the human workflow, and stays independent of whichever agent framework is in front of it. Three callers exercise the same `/api/v1/intercept` contract.

CALLER	HOW IT INTEGRATES	WHERE
OpenClaw (claude-cli runtime)	Shell wrappers per tool, plus an exec gate for	Deployed alongside the agent, not in this repo

	arbitrary commands. Each POSTs to <code>/intercept</code> and refuses to run unless the outcome is PERMIT.	
LangChain agents	<code>CertacitoGovernanceHandler.check_tool_call()</code> maps a tool call to an action type and returns a <code>GovernanceDecision</code> . Sync and async.	<code>backend/agents/langchain_interceptor.py</code>
Traffic simulator	Posts synthetic decisions under invented agent ids for demonstration.	<code>backend/services/simulator.py</code>

Two properties matter more than the integration mechanics:

- **The agent cannot opt out.** The wrapper is the only path to the underlying command, so bypassing governance means not running the tool at all.
- **Unreachable means denied.** If the API cannot be reached, the wrapper and the LangChain handler both return DENY (rule `FAILSAFE`). Taking Certacito offline stops the agent rather than freeing it, which is the opposite of how a monitoring system behaves and is the point of the design.

Agents authenticate with `X-API-Key` rather than a JWT, since there is no interactive login to perform. Unregistered agent ids are still evaluated and still audited; they simply do not appear in the registry.

Interception path

POST `/api/v1/intercept` is the core of the system. Order of operations:

1. **Semantic guardrail runs first.** Regex pattern sets for prompt injection, jailbreak and exfiltration attempts, each carrying a hand-tuned confidence. A proximity-based discount suppresses the common false positive where text *discusses* injection rather than attempting it. If the guard blocks it short-circuits: outcome DENY, rule `SEMANTIC-GUARD`, risk Critical, and it still writes an audit entry. The policy engine never runs.
2. **Policy evaluation**, wrapped in try/except. Any exception becomes DENY at Critical risk.
3. **Approval item created** on ESCALATE only.
4. **Audit entry written for every outcome**, PERMIT included. No path through this endpoint skips the audit write.
5. **WebSocket broadcast** so connected dashboards update without polling.

Fail-closed behaviour

Four independent points, all real:

CONDITION	RESULT
No policy rule matches the action	DENY, risk Medium, "denied by default"
Policy engine raises	DENY, risk Critical
Policy YAML missing at startup	rules stay empty, so everything hits the no-match DENY
No valid API key or JWT	401, request never reaches policy

The system defaults to refusing rather than allowing.

Policy engine

Rules load from `backend/policies.yaml` (12 rules) via the `POLICY_CONFIG` setting.

A further 10 pre-built rules ship in `rule_library.yaml` and can be imported into the database through the API, which is what populates the Policy Rules screen.

Conditions are parsed, not keyword-matched. The parser handles `==`, `!=`, `>`,

`IN`, `NOT IN`, `MATCHES`, `NOT MATCHES` and `AND`, resolved against named sets

(`approved_tools`, `approved_vendors`, `approved_domains`, `restricted_tables`, `restricted_datasets`) defined in the same file.

Where a condition cannot be resolved the rule **does not match**, and the request falls through to the default deny. That direction is deliberate: unresolvable conditions were previously treated as satisfied, which on a *permit* rule meant allowing exactly the thing the rule existed to gate.

The grammar covers `==`, `!=`, `>`, `<`, `>=`, `<=`, `IN`, `NOT IN`, `CONTAINS`,

`NOT CONTAINS`, `MATCHES`, `NOT MATCHES`, `AND`, `OR`, `NOT` and parentheses.

`OR` was genuinely missing at one point and this document said so; the

recursive-descent parser in `backend/services/condition_parser.py` supports it,

verified by the cases in `tests/backend/test_condition_parser.py`.

Audit chain

Append-only and hash-chained. Each entry's hash is SHA-256 over a preimage built

from every stored column: `entry_id`, `timestamp`, `agent_id`, `action_type`,

`outcome`, `risk_level`, `policy_rule`, `policy_desc`, `session_id`,

`payload_masked`, a `payload_hash`, and the previous entry's hash. The preimage is

sorted JSON rather than concatenated strings, so two different entries cannot

produce the same one by shifting where a field ends. The chain starts from a fixed

genesis hash.

`payload_hash` is taken over the **raw** payload, before masking. That is

deliberate - the integrity check then covers what the agent actually sent, not the

redacted copy we display. The masked copy is covered separately as its own field.

Covering every column matters. The hash previously covered 7 of the 12 stored

fields, so `risk_level` or the masked payload could be rewritten without breaking

the chain: a Critical could be dropped to Low and verification still reported

valid. `GET /api/v1/audit/verify` now walks from genesis and recomputes each hash

rather than only comparing stored hashes to one another.

Appends are serialised behind an `asyncio.Lock`. This is not incidental: two

concurrent intercepts could previously both read the same chain head and both

append to it, forking the log. `GET /api/v1/audit/verify` re-walks the chain server-side and recomputes every hash.

The guarantee is **tamper-evident, not tamper-proof**. Anyone with database access can still delete rows. What they cannot do is delete them without breaking the chain, and verification surfaces that. Append-only is enforced at application level; there is no database trigger.

Authentication and access control

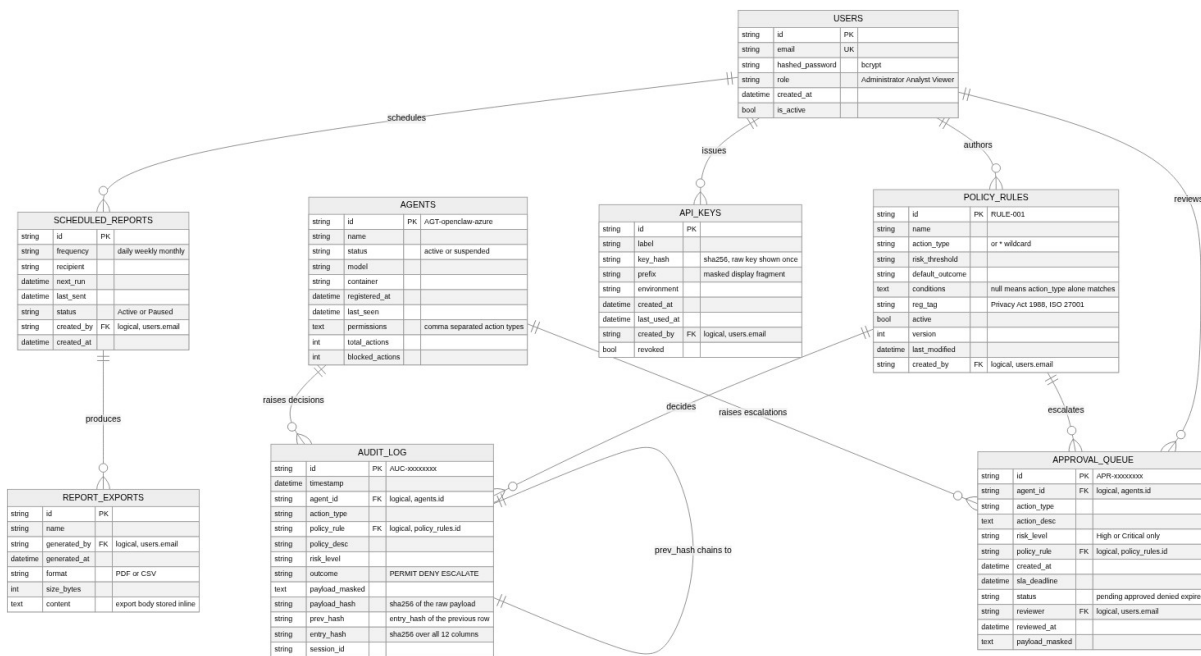
JWT bearer tokens for users, bcrypt for password hashing. `auth.py` calls bcrypt directly and truncates to 72 bytes itself. That used to go through passlib, which probes its backend with an over-length string on import - bcrypt 4.1+ raises on that rather than truncating, so it broke every login on a clean install and was worked around by pinning `bcrypt==4.0.1`. Dropping passlib removed the pin along with the stale-pin problem; the hash format is unchanged, so existing rows verify. Three roles: Administrator, Analyst, Viewer. Every endpoint carries an auth dependency; the role constants live in `backend/api/auth.py`. Agent-facing endpoints accept either an `X-API-Key` or a JWT via the `agent_or_user` dependency, because agents cannot perform the login flow. Payloads are masked before storage. Masking covers top-level keys only, which is recorded as a limitation.

Data model

PostgreSQL 16, accessed asynchronously through SQLAlchemy 2.0 with `asyncpg`.

Eight tables: `audit_log`, `approval_queue`, `policy_rules`, `users`, `agents`, `api_keys`, `scheduled_reports`, `report_exports`.

Entity relationship diagram, generated from the SQLAlchemy models in `backend/models/tables.py` (mermaid source in `docs/database-erd.html`). Dashed lines are logical relationships that the database does not enforce, explained directly below:



The relationships in that diagram are logical, not declared. No column carries a `ForeignKey` constraint, so the database will not reject an `audit_log` row naming an agent that was never registered. That is deliberate in one place and a known gap in the others:

- `audit_log.agent_id` **must** stay unconstrained. The whole point of the log is to record what an unknown or unregistered caller attempted. A foreign key here would make the system refuse to record exactly the events it exists to catch, and it would let anyone delete an agent row to orphan its history.
- `approval_queue.reviewer`, `policy_rules.created_by` and the `created_by` columns on `api_keys` and `scheduled_reports` hold an email rather than a user id, so a renamed account leaves the old string behind. These should be real references and are not.

Integrity is enforced in the application layer instead: the interception path resolves the agent before it writes counters, and the reviewer identity is taken from the verified JWT rather than the request body. Same register as the append-only guarantee below, which is also an application-level promise. Alembic is configured, but the schema is created by `Base.metadata.create_all()` on startup and the one revision in `backend/alembic/versions` is empty. Nothing in the deployment runs `alembic upgrade`. Recorded here rather than left implied. The agent registry was a dictionary in `backend/api/agents.py` until it moved into the `agents` table. Every restart of the container used to drop whatever had been registered through `POST /api/v1/agents` and put the hardcoded entry

back.

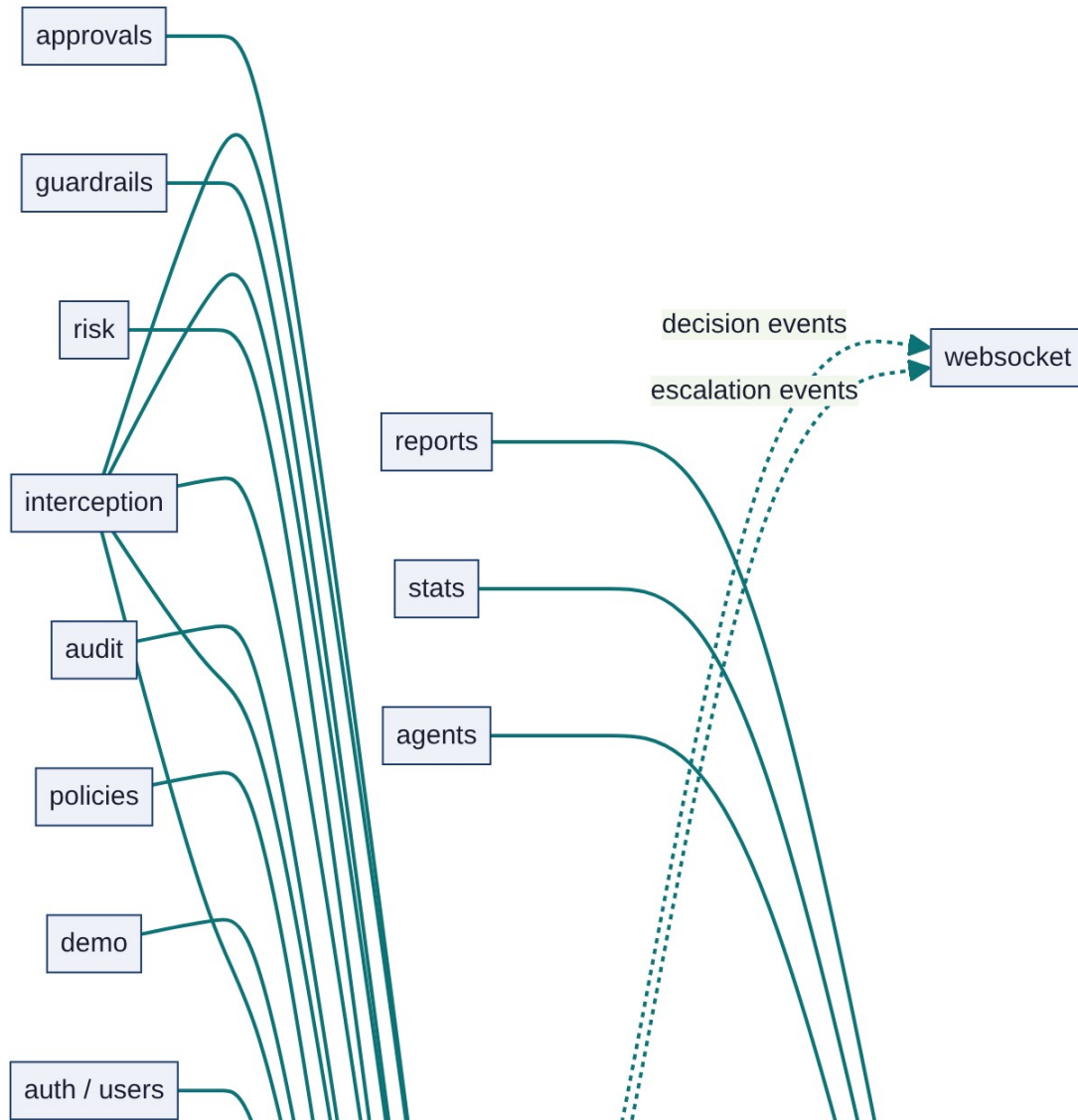
Redis 7 backs rate limiting and transient state.

A background scheduler task starts with the application and drives scheduled report generation.

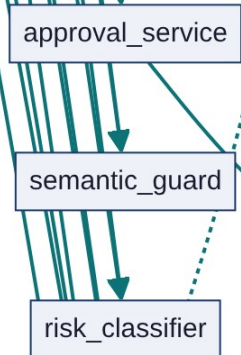
Component view

The fourteen API routers and the services they call. Dashed lines are event pushes rather than calls:

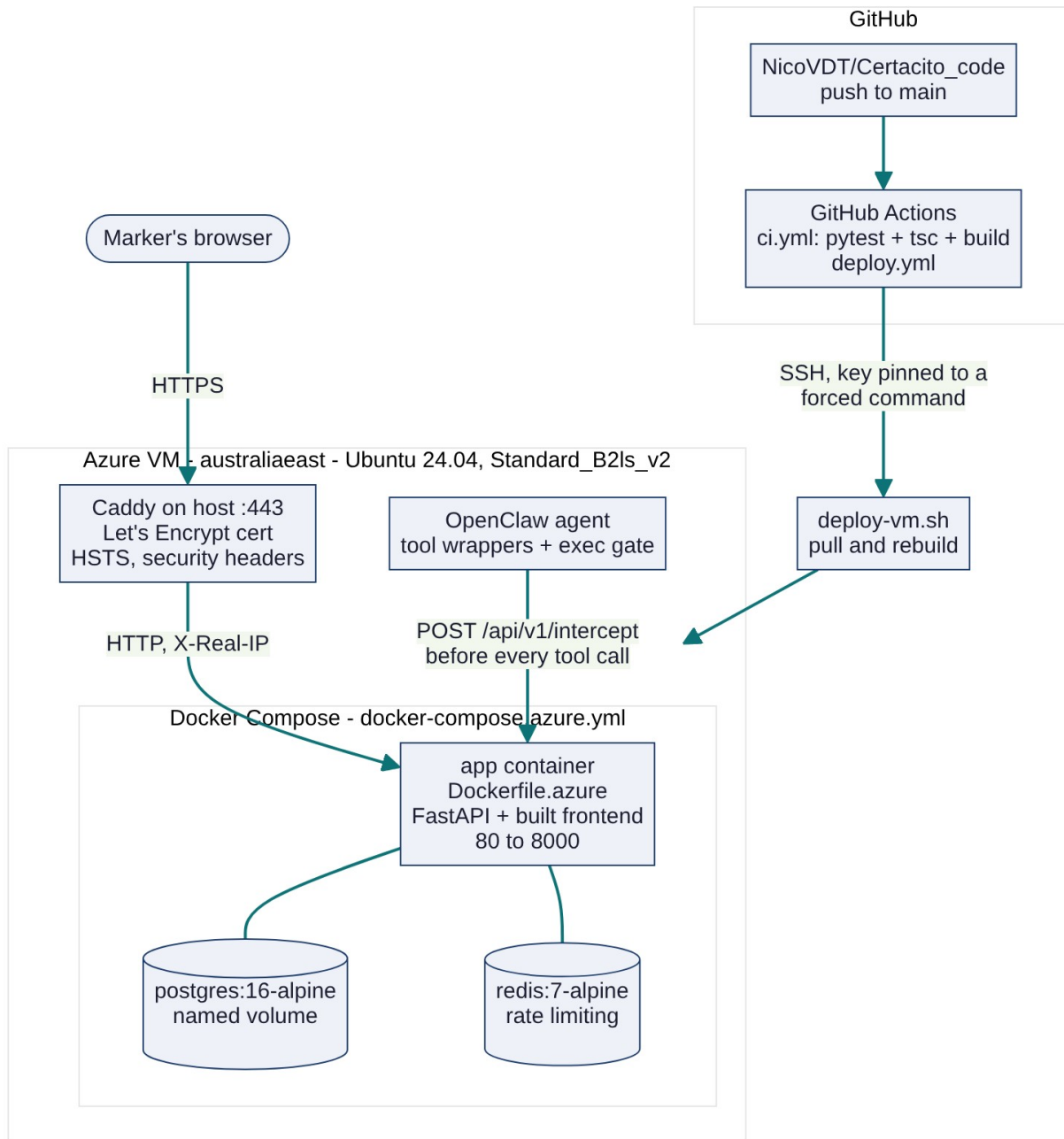
API routers - backend/api



Services - backend/services



Deployment



Production is Docker Compose on an Azure VM, not a managed platform. Three services from `docker-compose.azure.yml`:

- `postgres` (`postgres:16-alpine`) with a named volume
- `redis` (`redis:7-alpine`)
- `app`, built from `infra/docker/Dockerfile.azure`, published `80:8000`

The app image is a two-stage build. Stage one runs `npm ci` and `vite build`; stage two installs the Python dependencies and copies the built frontend in as `frontend_dist/`, which FastAPI serves. One image, one container, no nginx inside it.

TLS terminates in front of the container, in Caddy on the host

(`infra/caddy/Caddyfile`, deployed to `/etc/caddy/Caddyfile`). Caddy holds a real

Let's Encrypt certificate and reverse-proxies to the app container on port 80:

```
browser --HTTPS--> Caddy (host :443) --HTTP--> app container (:80 -> :8000)
```

The certificate is obtained without owning a domain by using `nip.io` wildcard DNS, which resolves `app.20-92-93-30.nip.io` back to the VM's public IP. Caddy adds HSTS, `X-Content-Type-Options`, `X-Frame-Options` and a referrer policy, and passes `X-Real-IP` through so the rate limiter buckets by real client rather than by whatever the client claims. The app trusts that header only when `TRUST_PROXY_HEADER=true` is set.

The bare-IP HTTP address still answers, because the container publishes port 80 directly, so anything pointed at the old URL keeps working. That is a known limitation rather than a design choice: a hardened deployment would redirect it or close it at the network security group.

Configuration is entirely environment variables (`DATABASE_URL`, `REDIS_URL`, `SECRET_KEY`, `AGENT_API_KEY`, `POLICY_CONFIG`, `RULE_LIBRARY`), read from a `.env` that is not in the repository. That makes the secrets externalisable to a secret manager without code changes.

Deployment is a pull-and-rebuild script on the VM, triggered over SSH by a GitHub Actions workflow. The CI deploy key is pinned to a forced command in the VM's `authorized_keys`, so it can run the deploy script and nothing else.

Non-functional characteristics

CONCERN	HOW IT IS ADDRESSED
Latency	WebSocket push rather than polling; sub-second dashboard updates
Integrity	SHA-256 hash-chained audit with server-side verification
Access control	JWT + bcrypt + RBAC on every endpoint
Confidentiality	PII masking on stored payloads (top-level keys)
Availability under load	Rate limiting, 60/min general and 10/min auth
Failure mode	Fail-closed at four independent points

Known gaps

Recorded here rather than only in the traceability matrix, because they are architectural rather than feature-level:

- Approval SLA expiry is evaluated lazily on read; there is no background expiry worker (FR-05).
- PII masking covers top-level payload keys only.
- Audit append-only is enforced in the application, not by a database constraint.
- CORS origins are still the development list.